CSI3007 - Advanced Python Programming

CASE STUDY

NAME : RAJAVELMURUGAN S

REG_NO : 23 MID0018

FACULTY : SHARMILA BANU K

LAB SLOT : C1

About symtable module:

The symtable module is a built-in module in Python's Standard Library that provides access to the symbol tables generated by the Python compiler. A symbol table is a special data structure that stores information about all the identifiers (such as variables, functions, classes, and imported modules) used in a Python program.

Unlike most Python modules, the symtable module does not execute the source code. Instead, it analyzes the code and collects information about the program's structure and scope. This process is called static code analysis.

The module helps developers understand how Python organizes variables and functions internally. It can determine whether a variable is local or global, identify function parameters, detect imported modules, and analyze nested functions and classes. Because of this, it is widely used in compiler development, code analysis tools, IDEs, linters, and debugging applications.

Why was the symtable Module Developed?

The symtable module was developed to help the Python compiler analyze Python source code before it is executed. It creates a symbol table, which stores information about identifiers such as variables, functions, classes, and their scope.

Before running a Python program, the compiler needs to understand:

- Which names are variables.
- Which names are functions or classes.
- Whether a variable is local, global, or a function parameter.
- Which modules are imported.
- How different scopes are related.

The symtable module provides access to this compiler information, making it useful for developers and tools that analyze Python code.

Objectives of Developing symtable

- To analyze Python source code without executing it.
- To generate symbol tables used by the Python compiler.
- To identify variables, functions, classes, and imports.
- To determine the scope of identifiers (local, global, parameter, etc.).
- To support static code analysis and debugging.
- To help build IDEs, linters, and code analysis tools.
- To improve code quality by detecting scope-related issues before execution.

Developer and Contributors of the symtable Module

The symtable module was originally developed by Jeremy Hylton, who is credited as the module author in the official Python documentation. He created the module to provide access to the symbol tables generated by the Python compiler.

The documentation for the symtable module was written by Benjamin Peterson, who is credited as the documentation author.

Over time, the symtable module has been continuously maintained and improved by the Python Core Development Team as part of the CPython project under the Python Software Foundation (PSF).

Role	Name
Module Author	Jeremy Hylton
Documentation Author	Benjamin Peterson
Maintained By	Python Core Development Team
Organization	Python Software Foundation (PSF)
Creator of Python	Guido van Rossum

Domain Applications of the symtable Module

Domain	Purpose	Examples
1. Static Code Analysis	Detects unused variables, undefined names, and scope-related issues before execution.	Pyflakes, Pylint, MyPy, Ruff
2. IDEs & Code Editors	Provides auto-completion, Go to Definition, Rename Symbol, and real-time error detection.	VS Code (Pylance/Pyright), PyCharm, Jupyter Notebook
3. Compiler Development & Education	Demonstrates how the Python compiler manages variables, functions, and scopes during compilation.	CPython Compiler, Compiler Design Courses
4. Code Refactoring Tools	Safely renames variables, extracts functions, and updates code while preserving scope relationships.	Rope, LibCST, Bowler

5. Security & Code Auditing	Helps analyze variable usage and data flow to identify potential security issues.	Static Application Security Testing (SAST) tools, CodeQL, Semgrep

Functions:

1. symtable.symtable()

Explanation

- Creates a symbol table from Python source code.
- Analyzes the code without executing it.
- Stores information about variables, functions, and classes.

Code:

import symtable

try:

code = "x = 10" # Python source code

table = symtable.symtable(code, "demo.py", "exec") # Create symbol table

print(table)

except Exception as e:

print("Error:", e) # Print error

```
import symtable

try:
    code = "x = 10"     # Python source code
    table = symtable.symtable(code, "demo.py", "exec") # Create symbol table
    print(table)
except Exception as e:
    print("Error:", e)   # Print error
```

```
<SymbolTable for module demo.py>
```

2. get_name()

Explanation

- Returns the name of the current symbol table.
- For the main module, it usually returns **top**.

- Useful for identifying the current scope.

Code:

```
import symtable

try:
    code = "x = 10"
    table = symtable.symtable(code, "demo.py", "exec")   # Create symbol table
    print(table.get_name())      # Show table name
except Exception as e:
    print("Error:", e)
```

```
top
```

3. get_type()

Explanation

- Returns the type of the symbol table.
- It tells whether the code belongs to a **module**, **function**, or **class**.
- In this program, it returns **module** because the code is at the module (top) level.

Code:

```
import symtable


try:
    code = "x = 10"
    table = symtable.symtable(code, "demo.py", "exec")
    print(table.get_type())       # Show table type
except Exception as e:
    print("Error:", e)
```

```
import symtable

try:
    code = "x = 10"
    table = symtable.symtable(code, "demo.py", "exec")
    print(table.get_type())       # Show table type
except Exception as e:
    print("Error:", e)
```

```
module
```

4. get_lineno()

Explanation

- Returns the starting line number of the current symbol table.
- It shows where the code block begins in the source file.
- For the main module, the starting line number is usually **0**.

Code:

```
import symtable


try:
    code = "x = 10"
    table = symtable.symtable(code, "demo.py", "exec")
    print(table.get_lineno())    # Starting line number
except Exception as e:
    print("Error:", e)
```

```
import symtable

try:
    code = "x = 10"
    table = symtable.symtable(code, "demo.py", "exec")
    print(table.get_lineno())    # Starting line number
except Exception as e:
    print("Error:", e)
```

```
0
```

5. get_identifiers()

Explanation

- Returns all the identifiers (names) used in the Python code.

- Identifiers include variables, functions, and classes.
- In this program, it returns the variable names **a** and **b**.

Code

import symtable

try:

code = """

a = 10

b = 20

"""

table = symtable.symtable(code, "demo.py", "exec")

print(table.get_identifiers()) # Show variable names

except Exception as e:

print("Error:", e)

```
import symtable

try:
    code = """
a = 10
b = 20
"""
    table = symtable.symtable(code, "demo.py", "exec")
    print(table.get_identifiers())     # Show variable names
except Exception as e:
    print("Error:", e)
```

```
dict_keys(['a', 'b'])
```

6. **lookup()**

Explanation

- Searches for a specific identifier (variable, function, or class) in the symbol table.
- Returns information about the specified symbol.
- In this program, it looks up the variable **x**.

Code

import symtable

try:

code = "x = 10"

table = symtable.symtable(code, "demo.py", "exec")

symbol = table.lookup("x") # Find symbol

print(symbol)

except Exception as e:

print("Error:", e)

```
import symtable

try:
    code = "x = 10"
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("x")       # Find symbol
    print(symbol)
except Exception as e:
    print("Error:", e)
```

```
<symbol 'x': LOCAL, DEF_LOCAL>
```

7. **get_children()**

Explanation

- Returns the child symbol tables of the current symbol table.
- Child symbol tables represent nested scopes such as functions or classes.
- In this program, it returns the symbol table for the add() function.

Code:

import symtable

try:

code = """

def add():

pass

"""

table = symtable.symtable(code, "demo.py", "exec")

print(table.get_children()) # Show child tables

except Exception as e:

print("Error:", e)

```
import symtable

try:
    code = """
def add():
    pass
"""
    table = symtable.symtable(code, "demo.py", "exec")
    print(table.get_children())   # Show child tables
except Exception as e:
    print("Error:", e)
```

```
[<Function SymbolTable for add in demo.py>]
```

```
import symtable

try:
    code = """
def add():
    pass
"""
    table = symtable.symtable(code, "demo.py", "exec")
    child = table.get_children()[1]   # Error: Only one child exists
    print(child)

except Exception as e:
    print("Error:", e)
```

```
Error: list index out of range
```

- There is only one child symbol table (add).
- Trying to access index 1 (the second child) causes IndexError: list index out of range.

8. **is_global()**

Explanation

- Checks whether a variable is a global variable.
- Returns **True** if the variable belongs to the global scope.
- In this program, x is a global variable, so it returns **True**.

Code

import symtable

try:

code = "x = 100"

table = symtable.symtable(code, "demo.py", "exec")

symbol = table.lookup("x")

print(symbol.is_global()) # Check global variable

except Exception as e:

print("Error:", e)

```
import symtable

try:
    code = "x = 100"
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("x")
    print(symbol.is_global())               # Check global variable
except Exception as e:
    print("Error:", e)
```

```
True
```

9. **is_local()**

Explanation

- Checks whether a variable is a local variable.
- Returns **True** if the variable is declared inside a function or local scope.
- In this program, a is a local variable inside the test() function, so it returns **True**.

Code

import symtable

try:

code = """

def test():

a = 10

"""

table = symtable.symtable(code, "demo.py", "exec")

```
    child = table.get_children()[0]          # Get function table
    symbol = child.lookup("a")
    print(symbol.is_local())                 # Check local variable
except Exception as e:
    print("Error:", e)
```

```
import symtable

try:
    code = """
def test():
    a = 10
"""
    table = symtable.symtable(code, "demo.py", "exec")
    child = table.get_children()[0]          # Get function table
    symbol = child.lookup("a")
    print(symbol.is_local())                 # Check local variable
except Exception as e:
    print("Error:", e)
```

```
True
```

10. **is_parameter()**

Explanation

- Checks whether a variable is a function parameter.
- Returns **True** if the variable is passed as an argument to the function.
- In this program, a is a parameter of the add() function, so it returns **True**.

Code

```
import symtable


try:
    code = """
def add(a, b):
    return a + b
"""
    table = symtable.symtable(code, "demo.py", "exec")
    child = table.get_children()[0]
    symbol = child.lookup("a")
```

print(symbol.is_parameter()) # Check function parameter

except Exception as e:

print("Error:", e)

```
import symtable

try:
    code = """
def add(a, b):
    return a + b
"""
    table = symtable.symtable(code, "demo.py", "exec")
    child = table.get_children()[0]
    symbol = child.lookup("a")
    print(symbol.is_parameter())            # Check function parameter
except Exception as e:
    print("Error:", e)
```

```
True
```

```
import symtable

try:
    code = """
def add(a, b):
    return a + b
"""
    table = symtable.symtable(code, "demo.py", "exec")
    child = table.get_children()[0]
    symbol = child.lookup("c")      # 'c' does not exist
    print(symbol.is_parameter())

except Exception as e:
    print("Error:", e)
```

```
Error: 'c'
```

- The function has only two parameters: a and b.
- lookup("c") searches for a variable named c, but it doesn't exist.
- Therefore, it raises an error.

11. **is_imported()**

Explanation

- Checks whether a name is imported from another module.
- Returns **True** if the identifier is imported using the import statement.

- In this program, math is an imported module, so it returns **True**.

Code

```
import symtable


try:
    code = "import math"
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("math")
    print(symbol.is_imported())             # Check imported module
except Exception as e:
    print("Error:", e)
```

```
import symtable

try:
    code = "import math"
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("math")
    print(symbol.is_imported())             # Check imported module
except Exception as e:
    print("Error:", e)
```

```
True
```

12. **is_namespace()**

Explanation

- Checks whether an identifier creates a new namespace.
- Returns **True** if the identifier is a class or function.
- In this program, Student is a class, so it returns **True**

Code

```
import symtable


try:
    code = """
class Student:
    pass
```

"""

table = symtable.symtable(code, "demo.py", "exec")

symbol = table.lookup("Student")

print(symbol.is_namespace()) # Check namespace

except Exception as e:

print("Error:", e)

```
import symtable

try:
    code = """
class Student:
    pass
"""
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("Student")
    print(symbol.is_namespace())            # Check namespace
except Exception as e:
    print("Error:", e)
```

```
True
```

13. **get_namespaces()**

Explanation

- Returns the namespace created by an identifier.
- It is used for identifiers that represent a class or function.
- In this program, it returns the namespace of the Student class

Code

import symtable

try:

code = """

class Student:

pass

"""

table = symtable.symtable(code, "demo.py", "exec")

symbol = table.lookup("Student")

print(symbol.get_namespaces()) # Get namespace list

except Exception as e:

print("Error:", e)

```
import symtable

try:
    code = """
class Student:
    pass
"""
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("Student")
    print(symbol.get_namespaces())  # Get namespace list
except Exception as e:
    print("Error:", e)
```

```
[<Class SymbolTable for Student in demo.py>]
```

```
import symtable

try:
    code = """
class Student
    pass
"""
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("Student")
    print(symbol.get_namespaces())

except Exception as e:
    print("Error:", e)
```

```
Error: expected ':' (demo.py, line 2)
```

- The class definition is missing a colon (:).
- Python cannot create the symbol table because of the syntax error.

14. **is_referenced()**

Explanation

- Checks whether a variable is used (referenced) in the code.
- Returns **True** if the variable is accessed after it is declared.
- In this program, x is used in print(x), so it returns **True**.

Code

import symtable

try:

```
    code = """
x = 10
print(x)
"""
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("x")
    print(symbol.is_referenced())  # Check if variable is used
except Exception as e:
    print("Error:", e)
```

```
import symtable

try:
    code = """
x = 10
print(x)
"""
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("x")
    print(symbol.is_referenced())  # Check if variable is used
except Exception as e:
    print("Error:", e)
```

```
True
```

15. **is_assigned()**

Explanation

- Checks whether a value has been assigned to a variable.
- Returns **True** if the variable is assigned in the code.
- In this program, x is assigned the value 100, so it returns **True**.

Code

```
import symtable


try:
    code = "x = 100"
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("x")
    print(symbol.is_assigned())       # Check assigned value
```

except Exception as e:

print("Error:", e)

```
import symtable

try:
    code = "x = 100"
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("x")
    print(symbol.is_assigned())       # Check assigned value
except Exception as e:
    print("Error:", e)
```

```
True
```

```
import symtable

try:
    code = "x = 100"
    table = symtable.symtable(code, "demo.py", "exec")
    symbol = table.lookup("x")
    print(symbol.is_assign())      # Wrong method name

except Exception as e:
    print("Error:", e)
```

```
Error: 'Symbol' object has no attribute 'is_assign'
```

- The correct method is **is_assigned()**.
- is_assign() does not exist, so Python raises an AttributeError.

Pros and Cons of the symtable Module

Pros

1. **No Code Execution Required**

   Analyzes Python source code without running the program.

2. **Built-in Module**

   Comes with Python's Standard Library, so no installation is needed.

3. **Scope Analysis**

   Identifies whether variables are local, global, parameters, or imported.

4. **Useful for Developer Tools**

   Helps build linters, IDEs, code analyzers, and debugging tools.

5. **Fast Code Analysis**

   Quickly creates symbol tables for source code.

6. **Supports Static Analysis**

   Helps detect coding issues before program execution.

7. **Easy to Use**

   Simple API with methods like lookup(), get_identifiers(), and is_global().

8. **Compiler Support**

   Uses the same symbol table information generated by the Python compiler.

Cons

1. **Does Not Execute Code**

   It only analyzes the code and cannot produce runtime results.

2. **Limited Functionality**

   Mainly focuses on symbol and scope information, not complete program analysis.

3. **Not Beginner Friendly**

   Requires knowledge of compiler concepts such as symbol tables and variable scopes.

4. **Python-Specific**

   Works only with Python source code.

5. **Cannot Detect Runtime Errors**

   It cannot identify errors that occur only when the program is executed.

References

1. **Python Software Foundation.** *symtable — Access to the compiler's symbol tables.* Python 3 Documentation. Available: https://docs.python.org/3/library/symtable.html
2. **Python Software Foundation.** *The Python Standard Library.* Available: https://docs.python.org/3/library/